

```

int main(){
    A a;
    B b;
    b.display(a);
    return 0;
}

```

ans -> provide easier solution to
real-world problems by implementing
real-world entities

- Object-oriented Programming ~
(OOPS)

type of example or
pattern or model

→ what is this??

→ (to design a program using objects and classes)

A programming paradigm (technique) in which things work around
an object → we need to include concept of object in our coding
technique

→ A type of entity having 2 things in all

↳ Having state or property (existence)

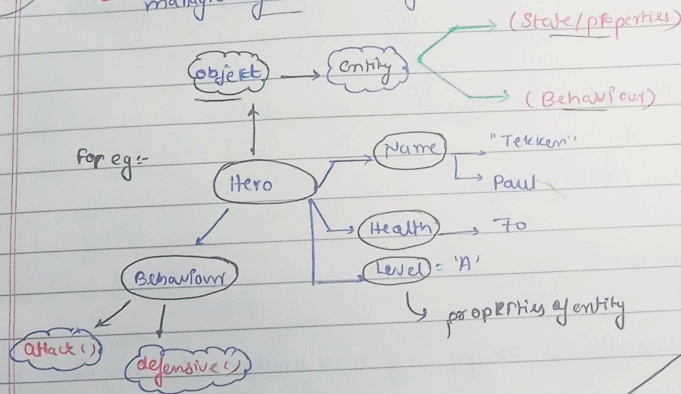
↳ A particular behaviour

In real life, like
microphone, lights,
laptops etc

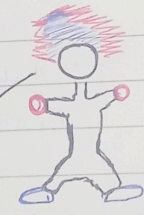
↓ In short, we want our programs to be related closer to real-world

But why??

oops → In order to increase readability of our program, increase its
managibility, extensibility → extensive nature to increase



oops isn't had my
friend!!



(Random)

Class: Hero

int health;

(Hero h1),
obj: h1

health = 100

garbage value with
size = 4 (same as size of
int as declared in a class)

Now, let's understand what is a class??

→ defines objects properties and functions
→ It is a user-defined datatype like
→ used for logical representation of data
int a; (integer) String str; (String datatype) char ch; (character)

→ like for = Hero (Ramesh)
(object) (user defined datatype)

* object → instance of a class Class: Hero

Implementation =

Student class been created

class student {

public: // access method

int id; // data member

int mobile;

String name;

properties of a class

int add (int x, int y) // member function

{

return (x+y);

}

}

→ A class does not occupy any space until or unless any memory space for object is been provided

→ object must be initiated to provide space to our created class.

We can create class file in the same file or linking with other file by using `#include "hero.cpp"`
↳ class name

DATE / /
Pg No. /

object being a runtime entity, this an instance of class → It can represent a person, place or any other item
↳ It can operate both data members as well as member functions

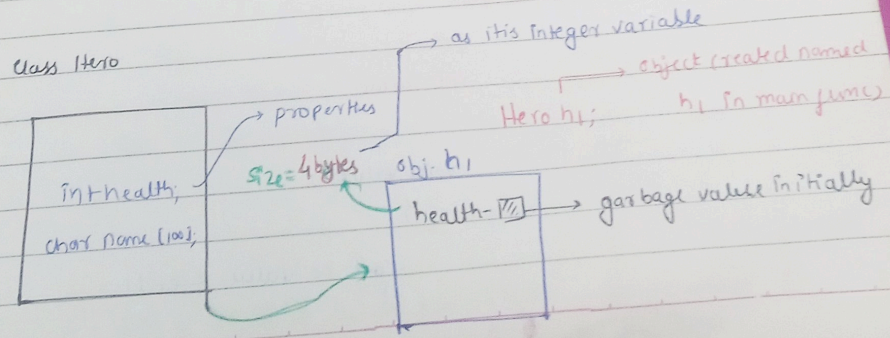
for eg. `Students = new Student();`
↓

(When object is created using new keyword, the space allocated to variable is a heap memory
↳ starting address being stored in stack memory

→ If new keyword is not been used to create an object, no space allocated in heap memory → object contains null value in stack

By chance, if we don't have any properties of created class, still after initializing our object, the class would have size of 1 byte by default
↳ (class doesn't occupy any space till the time an object is initiated)
→ no properties used

```
class Hero  
{  
    // public:  
}
```



Access properties / Data members -

↓
To access the data members of a class we can use • look operator

Hero h1;

cout << h1.health << endl;

cout << h1.strength << endl;

→, like this we can access properties

Access modifiers -

→ By default, it is private in C++.

to set the visibility and accessibility of class, methods, variables

↳ In short, it would describe how can we access the data members of a particular class

↳ Where all can we access these

within a class

outside a class

* Public

* Private

* Protected

→ If data members have been set to public (writing public:), we can access all of them anywhere (inside or outside a class)

→ All the properties below the class with access modifier as private can only be accessed within a class (not outside of a class)

↳ Class Hero {

private;

int value

void value()

{

cout << value << endl;

}

}

To set value via object -

```
main()
{
    hr.sum = 70;
}
```

this reduces (regards) scope of the variable
printing garbage value

- Protected access modifier → during the child class concept we would understand it clearly (functions and variables cannot be accessed outside class except child class)
- Concept of Getter/Setter →

To access properties (that are private) outside a class, we use these functions to access these members

```
class Hero
```

```
{
```

```
    int value;
```

```
    char ch;
```

```
    int get_value()
```

```
{
```

```
        return value;
```

```
}
```

```
    char getch()
```

```
{
```

```
        return ch;
```

```
}
```

```
void set_value (int value, char name)
```

```
{
```

// we can also give condition

```
if (name == "A")
```

```
{
```

```
    value = value;
```

```
}
```

```
}
```

getter function

value stored in a variable in private class

```
main()
```

```
{
```

```
    Hero ramesh;
```

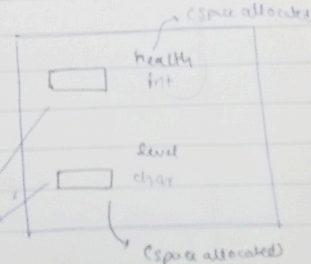
```
    ramesh.set_value (70, 'A');
```

```
    cout << ramesh.get_value ();
```

```
}
```

size = (4+1) = 5 bytes

Behind the Scenes:-



but on compiler it would give = 8 bytes

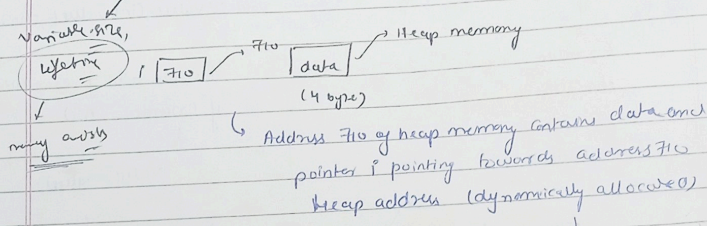
(due to padding and greedy alignment)

memory is automatically managed
 ↳ allocated memory as program starts and deallocated as program ends

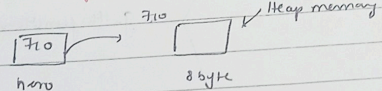
Static vs dynamic allocation:-
 ↳ size and type of memory is fixed and known at compile time
 ↳ Allocating the data, same time as declaration and not storing any pathway (address where actual data is stored)

Stack memory used
 ← `int a = 10;` // static allocation → compile time

`int *i = new int;` // dynamic allocation → runtime



Some we can do for classes - `Hero *h = new Hero;` new keyword used to allocate or de-allocate → depends on program



`b → getSet value (710);` `b → getHealth (100, 'A');`

for output `cout << (*b) = level << endl;`
`cout << (*b) = getHealth() << endl;`
 ↳ difference operator
 ↳ potential memory leaks or overhead issue

or
`cout << b → getHealth() << endl;`
`cout << b → level << endl;`

↳ for dynamic allocation → size can be determined during execution of program providing flexibility

Constructor → original class remains automatically
Invoked during declaration

DELTA
Date / /
Pg No.

* Concept of Constructor and Destructor -

(Invoked automatically at time of object creation)

Whenever we create an object, 1st thing that is being called is "constructor"

For eg Hero ramish,

• Invoked during object creation time



Behind the scene a default

• No return type

constructor without any parameter

• No i/p parameter (for

ramish. Hero() been
made

eg default constructor)

• Same name as the class

and may be defined inside or
outside class

↳ When we try to write our own constructor, the constructor that
had already been created by system will be replaced
by this

↳ no argument, no parameters

→ constructor can be overloaded

class Hero {

// constructor (default with no parameters) created

Hero()

default
constructor

```
{  
    cout << "constructor made" << endl;  
}
```

* No virtual constructor can be declared

→ Parameterized Constructor:-

↳ constructor having parameters with it while declaration



Hero (int value, char* ch)

↳ constructor with parameters

refers to the current instance of a class
 + can be used to pass an object as parameter to another method
 + can be used to refer to current class instance variable

→ concept of this keyword

while allocating value from parametrized constructor to variables in our class

if the parameter name = variable name then this might lead to sort of confusion

∴ we use this keyword to separate both of them

```

For eg      int health;           (variable been assigned value
            this (int health)      of local of parameter)
            {
                this → health = health; // health = health;
            }
  
```

distinguish
 class parameter
 and object
 constructor
 parameter

this is a pointer which can point towards particular object variable (current object)

↳ it stores current object's address

* Concept of copy constructor:-

will do after some concepts of inheritance etc.
 (4 pillars of OOPS)

+ Inheritance - It's a process in which one object acquires all the properties and behaviours of parent's object automatically
 (acces properties and behaviour from another class)
 ↳ we can reuse, extend or modify attributes and behaviours that are defined in other classes

* Reuse, modify the attributes

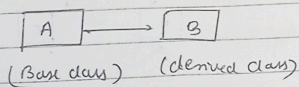
DELTA
Date / /
Pg No.

- Parent object → Base class
- child object → derived class

C++ Syntax: (sub class) (parent class)
 class derived class :: access modifier base class
 (For eg) class Dog :: public pet

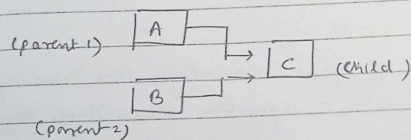
(i) Single inheritance - when 1 class inherits properties from 1 base class only

(Derived class = Base class properties + own properties)

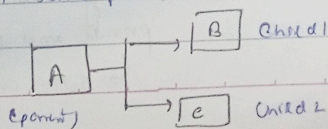


(ii) Multiple inheritance - deriving a new class that inherits properties from 2 or more base class

(class Pet = public Dog, public cat)

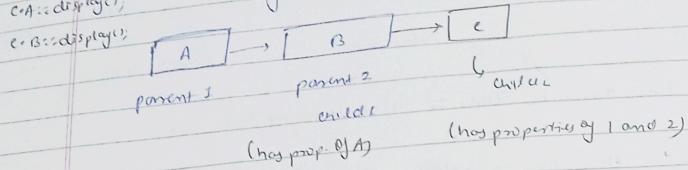


(iii) Hierarchical inheritance - A type of inheritance where a base class can produce 2 or more child classes (child classes can inherit properties from same parent)

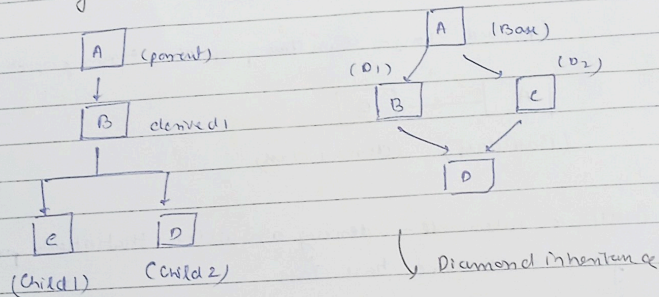


* Ambiguity resolution → In case of multiple inheritance, if 2 classes have same member function (eg. void display) in both B and C → usage of scope resolution operator ::

(iv) Multilevel inheritance — multiple inheritance is a process of deriving a class from another derived class
(grand parent — parent — child comp)



(v) Hybrid inheritance — This a combination of various types of inheritance → simple, multiple and hierarchical
(multipath inheritance)
(Single + multiple) or (multiple hierarchical)



* Encapsulation

→ Wrapping up the data and information into single unit
process of combining data and functions into a single unit, called class. In encapsulation, data is not accessed directly → accessed through functions present inside the class

capsule

↓
attributes of class are kept private and public
→ setter and getter methods

data-binding - process of binding the application UI and business logic → any changes in business logic will reflect in application UI

binding the data together and function that manipulates data

encapsulation → responsible for hiding the data

restricting methods to access objects

Securitized, reduces human error

→ only high level access, no explicit reveal of below level

* Abstraction - providing abstract detail view, model or structure of a real life problem to reduce unnecessary details

↳ providing necessary information regarding the data only and hiding unnecessary details like (background details and information)

* Polymorphism - This is the ability to present the same interface for differing underlying forms (data types)

↳ every class has different underlying data

↳ for eg → a shape { can be a square or a circle

(many forms)

Types of polymorphism

- Compile Time (Static)
- Runtime (Dynamic)

(eg- method overloading)

↳ implemented at compile time
function call resolved at compile time

(Runtime polymorphism)

↳ also called as dynamic polymorphism

→ for eg function overriding

↳ when the child class contains method

that's already present in parent class

(child class overtakes (overrides) parent class)

↳ parent and child class with same function name with different definitions

call of function is at runtime

→ method overloading - method overloading is a technique which allows o/s to have more than 1 function with same function name but different functionality

↳ how can it vary??

- return type of function
- type of parameters passed to fun
- no. of parameters passed to fun

```
int add (int x, int y)
{
    return (x+y);
}
```

```
int add (int x, int y, int z)
{
    return (x+y+z);
}
```

Concept of Copy Constructor -

A copy constructor is an overloaded constructor used to declare and initialise an object from another object.

↓

default copy constructor user defined copy constructor

C++ Sample code:-

↓

it is a member function which initializes an object using another object of same class

```
#include <iostream>
using namespace std;
class go {
```

Sample code

```

public:
    int x;

    go(int a)
    {
        // parameterized
        x = a;
    }

    go ( go d )
    {
        // copy constructor
        x = d.x;
    }
};

int main()
{
    go a1 (20);
    go a2 (a1);
    cout << a2.x << endl;
}
    
```

* Destructor - works opposite to constructor, destroys object of class
 ↳ can be defined only once in class
 (an object when created by constructor and destroyed when created)

~ = tilde sign for destructor
 ↳ having same name as constructor

```

~A()
{
    cout << "Destructor in use" << endl;
}
    
```

↳ To access private members of other class acting as its friend
 * Friend function - Acting as a friend of the class. It can access the private and protected members of the class

↳ not a member of the class but must be listed in class definition
 ↳ can access and manipulate the data

friend function - has the ability to access private data of class

- friend function cannot access private member directly
- ↳ has to use an object name and dot operator with each member name
 - ↳ uses objects as arguments

```
eg friend int mul(A &k)
{
    return (k.a * k.b);
}
```

- ⇒ Aggregation → process in which one class defines another class as entire reference
- ↳ another way to reuse the class

~ Virtual function in C++ ~

It is a member function present in base class and redefined by the derived class

- ↳ when we use the same function name in both base and derived class, function in base class is declared by keyword virtual

- ↳ Virtual functions cannot be static

→ It may have virtual destructor but cannot have virtual constructor

// virtual function (redefined in derived class)

```
virtual void print()
{
    cout << " print base class" << endl;
}
```

bptr → print(); // virtual function values printing

↳ topics

++

* Pure Virtual function - It is not used for performing any task
↳ only serves as a place holder

↳ we can have → A function declared in base class having no definition relative to base class

Implementation

but use must

try to override it

↳ A class containing pure virtual function cannot be used to declare objects on its own

↳

Known as abstract base classes

eg - Virtual void show() = 0 → example of pure virtual function
// pure virtual function

~ Abstract Classes ~

↳ Sometimes implementation of all fun cannot be provided by base class

In C++, class is made abstract by declaring at least one of its functions as a pure virtual function (at least)

↳

pure virtual function is specified by placing "= 0" in its declaration → implementation must be provided by derived classes

* Concept of destructor in OOPS-

It is also a special member function as a constructor with same name as a constructor. It is used to destroy the class objects created by the constructor.

↳ it has a tilde (~) sign in addition

⇒ It is automatically been invoked by compiler when its corresponding constructor goes out of scope and release the memory space that is no longer required by program.

↳ destructor never require any arguments nor return any value is cannot be overloaded

↳ should be declared in public section

```
#include <iostream>
```

```
using namespace std;
```

```
class num {  
public:
```

```
    ~num()  
    {
```

```
        cout << "Destructor been invoked" << endl;  
    }
```

```
}
```

No of times object created

↳

no of times destructor has been called

* Abstraction continued -

- Helps user to avoid writing low level code
- Avoids code duplication and increases reusability
- Helps to increase security of an application or program as only required details are provided to the user.

For eg- Bank → we just need to put details no use of internal complete functionality

// example code:-

```
#include <iostream>
using namespace std;
```

```
class ImplementAss
```

```
{
    private:
        int a, b;
```

```
public:
```

```
void set (int x, int y)
{
```

```
    a = x;
```

```
    b = y;
```

```
}
```

```
void display ()
```

```
{
    cout << "a = " << a << endl;
```

```
    cout << "b = " << b << endl;
```

```
}
}
```

```
int main()
{
```

```
    ImplementAss obj;
```

```
    obj.set (10, 20);
```

```
    obj.display ();
}
```

* Polymorphism - It means having many forms
↳ occur when there is a hierarchical mode inheritance

↳ A call to a member function will cause a different function to be executed depending on the type of object that invokes the function

→ allows us to reuse the code by creating one function that is usable for multiple uses

↳ for eg - Creating a function that can add both strings as well as both numbers

* Compile time polymorphism → can be achieved in 2 ways

i) Function overloading -

when there are multiple functions with the same name but take different parameters as arguments → then these functions are said to be overloaded

↳ functions can be overloaded by changing no. of arguments and type of arguments

→ code example

On Next Page ⇒

```
#include <iostream>
using namespace std;
class Check
{
public:
    void func()
    {
        cout << " no argument" << endl;
    }

    void func(int x)
    {
        cout << " function with one argument" << endl;
    }

    void func(int x, int y)
    {
        cout << " function with 2 arguments" << endl;
    }
}

int main()
{
```

```
    Check obj;
    obj.func();
    obj = func(4);
    obj = func(4, 7);
}
```

2. Operator Overloading.

C++ also provides the option to overload operators. So a single operator '+' when placed b/w 2 integers adds them and if placed b/w strings

Concatenates them.

Eg. Complex no.
12 + 19

DELTA
Date / /
Pg No

⇒ Runtime polymorphism - It is also known as dynamic polymorphism or late binding

↳ function call is resolved at runtime
→ achieved by virtual function or function overriding

* Virtual function

↳ virtual is a key word

◦ A virtual function is a member function in the base class that we can expect to redefine in derived classes

→ when defined in base class, then on runtime basis of object assigned, respective class function is called

```
#include <iostream>
using namespace std;
```

```
class Base
```

```
{
```

```
public:
```

```
virtual void print()
```

```
{
```

```
cout << "Base class function" << endl;
```

```
}
```

```
};
```

```
class Derived: public Base
```

```
{
```

```
public:
```

```
void print()
```

```
{
```

```
cout << "Derived class function" << endl;
```

```
}
```

```
};
```

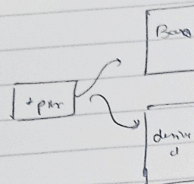
```
int main()
{
    // creating a base class pointer to point individual classes

    Base *bptr;

    Base base;
    Derived derived;

    bptr = &base;
    bptr->print(); // prints Base class

    bptr = &derived;
    bptr->print(); // prints derived class
}
```



→ Friend function implementation → Look at github repository